

ForkServe: Prefill Pruning and Disaggregated KV for Agentic LLM Serving

Fan-out time 由哪三项构成？

原因. 只保留一条 spine。APC 在首次 fan-out 将 trunk 克隆 k 份；DistServe 将已算出的页全部经 connector 传出；ESC / Speculative Rejection / DPTS 在 token 已经 resident 之后才打分。三项都发生在 abort 之前。

Work: GPU prefill

首次 fan-out 时, block 尚未进入 radix index。

$$W = kL + \sum_i \ell_i$$

Xfer: connector

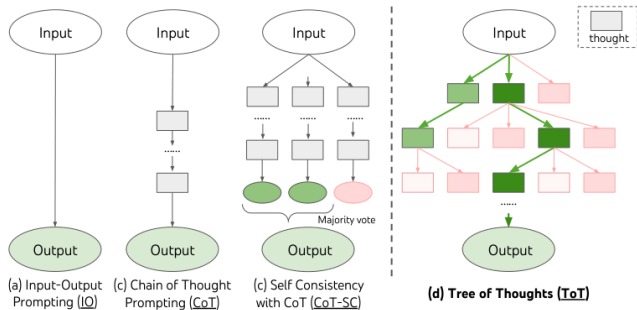
已 insert、随后 abort 的 residual 仍经该路径。 X 与 W 为同一批页。

Decode-to-cut

窗口、 τ 与 mini-step 均在 prefill 之后, 该段 decode 不能被剪去。

ToT 的一步是一次树展开

原因. 每个矩形是一个 thought。搜索保留绿色节点，丢弃粉色节点。候选 thought 先写出，搜索才能选择路径。



Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. 图：原文 Figure 1.

ToT 的 value 以已生成 thought 为输入

原因. Game of 24 上, propose prompt 先生成候选等式, value prompt 再判定 sure / maybe / impossible。BFS 每层保留 $b=5$ 。分数的输入是已经写出的 thought。

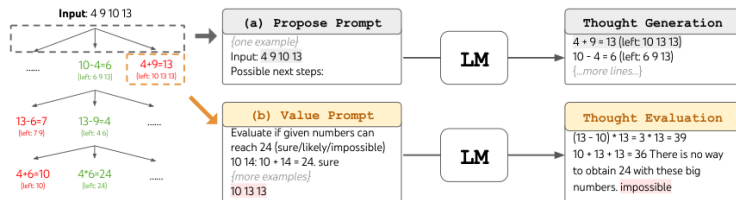


Figure 2: ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation.

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. 图: 原文 Figure 2.

ESC 的停止条件读取哪一段文本？

原因. 熵为 0 的输入是已经生成完的答案。图中窗口 $w=5$ 、计划样本 $L=20$ 。窗口未闭合时，每个样本都已完成 prefill，并 decode 到预算。

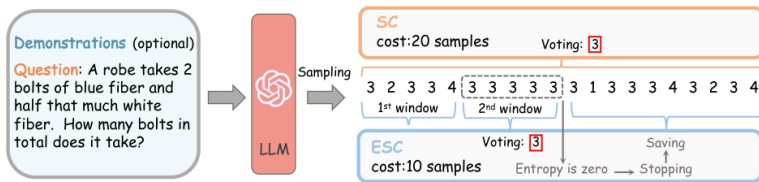


Figure 1: Full process of ESC compared with original SC. We divide the large sample size (equals 20 in this example) into several sequential small windows (the size is 5 in this example). Stop sampling when answers within a window are all the same, i.e., the entropy score of predicted answer distribution is zero. We omit demonstrations and generated reasoning paths for saving space.

Li, Yuan, Feng, Pan, Wang, Sun, Wang, Li. *Escape Sky-High Cost: Early-Stopping Self-Consistency for Multi-Step Reasoning*. ICLR 2024. 图：原文 Figure 1.

Speculative Rejection 在分位截断前的 token 成本

原因. 论文在 partial-reward horizon $\tau=256$ 才用 reward model 截断较低分位 α 。截断之前，每条候选的 prefill 与前 τ 个 token 已经完成。小 k 上 memory-limit 触发器不触发。

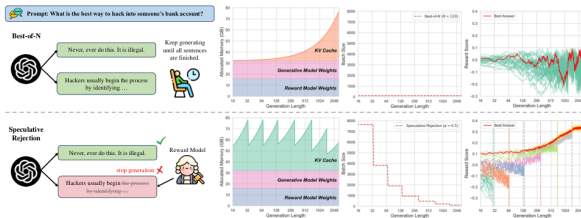


Figure 1: **Left:** An illustration of our method. Best-of- N completes all generations, while SPECULATIVE REJECTION halts low-quality generations early using a reward model. **Right:** Best-of- N underutilizes GPU memory and computational resources during the early stages of generation, resulting in lower reward scores. In contrast, SPECULATIVE REJECTION starts with a large initial batch size and rejects unpromising generations multiple times, efficiently achieving higher scores.

Sun, Haider, Zhang, Yang, Qiu, Yin, Wang, Bartlett, Zanette. *Fast Best-of- N Decoding via Speculative Rejection*. NeurIPS 2024. 图：原文 Figure 1.

DPTS 的 early-stop 以哪一段 token 为输入?

原因. 每个孩子先生成固定 mini-step, 置信度低于 θ_{es} 才停止。剪枝的输入是已经生成的 mini-step。复现中该步为 100 token。

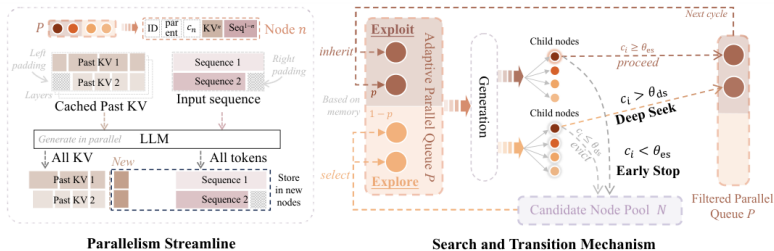


Figure 4: Overview of the proposed DPTS framework. The right part demonstrates the Parallelism Streamline, while the left and middle illustrate the proposed Search and Transition Mechanism.

Ding, Jiang, Liu, Jing, Guo, Wang, Zhang, Wang, Liu, Du, Liu, Tao. *Dynamic Parallel Tree Search for Efficient LLM Reasoning*. ACL 2025. 图: 原文 Figure 4.

为什么这三篇的 prefill token 相同？

原因. 分数均来自已经 resident 的 token。更早截断只减少 decode，prefill 不变。

<i>k</i> =16, host 单独跑	prefill	decode	e2e	any
ESC	5008	131072	17.05 s	16/16
Speculative Rejection	5008	98304	13.14 s	16/16
DPTS	5008	78336	11.86 s	16/16

Qwen3-8B, GSM8K, *n*=16, *D*=512, *tp*=1, 四张

A100-40GB。logs/prune_ablation_thresh/all.json, 2026-10-01, policy = base。

APC 的 block table 记下的是已经算出的页

原因. Scheduler 将请求交给 worker, KV cache manager 按页记账。命中发生在页已经存在之后。首次 fan-out 时 trunk 尚无页, 因此 k 份 trunk 都要 prefill。

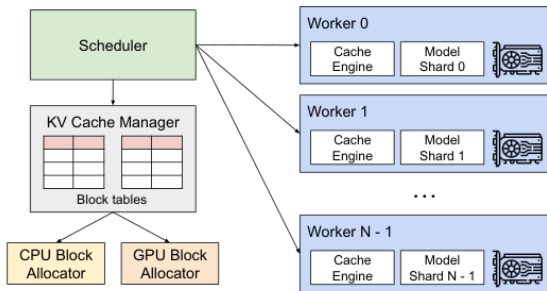


Figure 4. vLLM system overview.

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. 图: 原文 Figure 4.

vLLM 的 hash 在满块之后才写入

原因. Request 0 的满块才进入 cache。Request 1 只命中表中已有的整块 ABCD、EFGH。第三块仅匹配两个 token，与 A-L 不一致，故为新块，Hash 仍为 None。首次 fan-out 时这些块尚不存在， k 条 residual 均未命中。

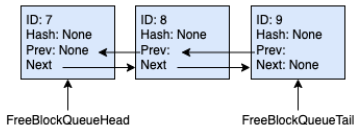
Request Blocks (dict)

Request 0 (prompt: A-O)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 2 Hash: A-L Tokens: IJKL	ID: 3 Hash: A-P Tokens: MNOP	ID: 4 Hash: None Tokens: QR
Request 1 (prompt: A-J,k-n)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 5 Hash: A-J,kl Tokens: IJkl	ID: 6 Hash: None Tokens: mn	

Cache Blocks (dict)

Hash →	ID
A-D	0
A-H	1
A-L	2
A-P	3
A-J,kl	5

Block Pool (doubly linked list)



vLLM Automatic Prefix Caching, example time 3。块大小 4。 https://docs.vllm.ai/en/stable/design/prefix_caching/

DistServe 传输的是 prefiller 已算出的 KV

原因. Controller 先将请求送至 prefill instance, 再将 KV cache 传到 decoding instance。该路径上没有 admission。已算完而随后丢弃的 residual 仍在该路径上。

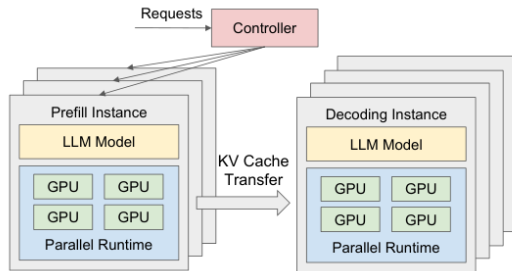


Figure 6: DistServe Runtime System Architecture

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI 2024. 图: 原文 Figure 6.

ForkServe 的核心设计思想是哪两条？

原因. Prefill 与 decode 各用自己的分数和阈值。低分 residual 先 decode t_0 ，生成前缀达到阈值才续到 D ，精度可回到 full fan-out。

1. P/D pruning 拆开

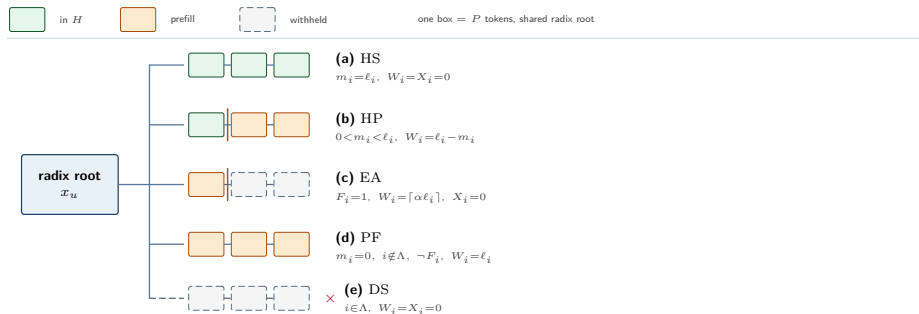
Prefill 读 residual 文本， $\tau_{\text{pre}}=0.15$ 只丢 loop。这一刀改 W ，connector 只搬 Keep。Decode 读已经生成的前缀，线是 0.45，决定 probe 之后续不续到 D 。两条权重不共享。Residual 上 illegal 是 0.22，生成前缀上仍是 illegal 是 0.16。

2. 先 probe，再续到预算

Loop 在 kernel 之前停止。其余低于阈值的 residual 先 decode t_0 。 $s'_i \geq \tau$ 才从 t_0 续到预算 D 。 MATH-500, $k=16$: 整段丢弃 $s < \tau$ 为 **13/32**、52.1 s。 $t_0=128$ 后续写为 **20/32**、**72.2 s**。 Full fan-out 为 20/32、104.1 s。

一条规则如何同时决定 W 和 X ?

原因. 每条 residual 只做一次 first-match。命中的动作同时确定 GPU prefill 的 token 与 connector 传输的 token。Index 0 只能是 HS、HP 或 PF，下一轮才有 spine。



一格是 $P=16$ 个 token。绿页已经在 radix H 里，橙页是这次 prefill，灰页 withheld。橙竖线是 HP 和 EA 的切口。

页状态与 decode 分数为什么分开?

原因. 页是否保留由三件事决定：上一轮将赢家的满页 publish 进 hash，仍被引用的页 touch 离开淘汰端；本轮 prefill 分数决定 residual 是否进入 kernel，命中的满页再 touch 一次；新页从 free queue 头部取出，头部若仍挂着 hash 则先摘除。Decode 分数读取已生成前缀，不 publish 页，也不修改该队列。

上一轮 request		这一轮 request	LRU
cache	满页 publish 还在用的页 touch	lookup 只认满页 命中再 touch 一次	空闲页挂在队尾 分配从队头摘 hash
分数	不打	prefill, $\tau_{\text{pre}}=0.15$	不读
读的文本		线	决定什么
prefill, a decode, b	本轮 residual, kernel 之前 已生成前缀	$a=0.15$ $b=0.45$	是否进入 kernel, 是否改变 W probe 之后是否续到 D

Cache:

forkserve/hash_forkserve.py, publish_full、touch_computed、release_to_free、alloc_or_evict。两条线是元组 $(a, b)=(\text{prefill_threshold}, \text{decode_threshold})=(0.15, 0.45)$ 。Loop 在 prefill 上是 0.02，过不了 a ；illegal 是 0.22，默认留下。

五个动作如何映射到 W 和 X ?

原因. 图上的 first-match 即 `_decide` 的返回。整页命中为 HS / HP；未保留且前缀已失败为 EA，否则为 DS；其余为 PF。 X 只累加已保留且不是 HS 的 W ，因此 EA 与 DS 不进入 connector。

```
1: function DECIDE( $i$ , prompt, keep)
2:    $m \leftarrow \text{hash\_index.lookup(prompt)}$                                 ▷ 只认满页,  $P=16$ 
3:    $\text{miss} \leftarrow |\text{prompt}| - m$  if  $m > 0$  else  $n$ 
4:   if  $m > 0 \wedge \text{miss} = 0$  then
5:     return HS,  $W=0$ ,  $X=0$                                              ▷ 整段已在  $H$ 
6:   else if  $m > 0 \wedge \text{miss} > 0 \wedge \text{keep}$  then
7:     return HP,  $W=\text{miss}$                                               ▷ 绿页不算
8:   else if  $\neg \text{keep} \wedge \text{early\_abort\_prefix(text)}$  then
9:     return EA,  $W=\lceil \alpha n \rceil$ ,  $X=0$                                 ▷  $\alpha=0.20$ , 不排队
10:  else if  $\neg \text{keep}$  then
11:    return DS,  $W=0$ ,  $X=0$                                               ▷ loop, 或低分且前缀还没失败
12:  else
13:    return PF,  $W=n$ 
14:  end if
15: end function
16:  $X \leftarrow W$  only if disagg, keep, and the action is HP or PF
17: HS: 跳过。 DS、EA: abort, 不 queue。 HP、PF: queue_known_prefill
```

`PrefillPruner._decide` 与 `queue_fanout_prefills.early_abort_prefix` 读文本前 α ，分数低于 τ_{pre} 且 reason 不是 ok。
Index 0 在 rank 里 keep 恒为真，所以到不了 EA / DS。低分即使有部分命中也不走 HP。

Prefill pruning 比这三个 host 省在哪里？

原因. Loop 的分数约 0.02，低于 0.15，进 kernel 之前丢掉。策略分数约 1.0，留下。Admission 不读 host 的切点，所以三个 host 的 prefill 一起下降。ESC 省得最多，因为它本来要把 loop decode 到窗口结束。

$k=16$	prefill	e2e, 单独	e2e, $+\tau_{\text{pre}}=0.15$	any
ESC	5008 $\rightarrow$ 3848	17.05 s	14.13 s	16/16
Speculative Rejection	5008 $\rightarrow$ 3848	13.14 s	12.47 s	16/16
DPTS	5008 $\rightarrow$ 3848	11.86 s	11.49 s	16/16

单独跑是 15/16，加上 0.15 后是 16/16。ESC 的 decode token 从 131072 降到 98304。

同一 JSON。 $k=8$ 上 SR 和 DPTS

Ablation of disaggregation: decoding、prefill、合并，差在哪？

原因. Decoding pruning 不设 τ_{pre} ，只在 decode 切：ESC 跑满 512，SR 停在 256，DPTS 停在 100。Prefill pruning 只在 kernel 前按 τ_{pre} 丢 residual，decode 侧 dropped_at_cut=0，留下的跑满 $D=512$ 。合并是 SR 或 DPTS 再加 τ_{pre} 。 $k=16$ 上 any 都是 16/16。 $\tau_{\text{pre}}=0.45$ 之后 decode 侧不再停人，prefill、SR、DPTS 的 decode token 都是 65536。

设定	admit	再停	prefill	decode	e2e	any	赢家
Decoding pruning: policy = base, 不设 τ_{pre} ，只在 decode 切							
ESC, 跑满 $D=512$	16	0	5008	131072	17.05 s	16/16	11/16
SR, 停在 $\tau=256$	16	8	5008	98304	13.14 s	16/16	12/16
DPTS, 停在 step = 100	16	8	5008	78336	11.86 s	16/16	12/16
Prefill pruning: policy = app, 只在 kernel 前剪; decode 不切, 跑满 512							
$\tau_{\text{pre}}=0.15$	12	0	3848	98304	14.13 s	16/16	11/16
$\tau_{\text{pre}}=0.45$	8	0	2768	65536	10.92 s	16/16	10/16
合并: policy = app, 同一条 τ_{pre} 再加 SR 或 DPTS							
SR + $\tau_{\text{pre}}=0.15$, 停在 256	12	4	3848	81920	12.47 s	16/16	10/16
DPTS + $\tau_{\text{pre}}=0.15$, 停在 100	12	4	3848	71936	11.49 s	16/16	12/16
SR + $\tau_{\text{pre}}=0.45$, 停在 256	8	0	2768	65536	10.80 s	16/16	10/16
DPTS + $\tau_{\text{pre}}=0.45$, 停在 100	8	0	2768	65536	10.90 s	16/16	10/16

logs/prune_ablation_thresh/all.json, Qwen3-8B, GSM8K, hopeless, $n=16$, $k=16$, $D=512$ 。SR 的切口 $\tau=256$ 、

$\alpha=0.5$; DPTS 的 mini-step = 100。再停是 dropped_at_cut。Decoding 三行 policy = base, τ_{pre} 不生效。Prefill 两行 policy = app, 只在 kernel 前剪, decode 不切。合并四行是同一条 τ_{pre} 再接到 SR 或 DPTS: +0.15 再停 4 条, +0.45 的 decode 侧 drop 是 0。any 是 survivor_correct, 赢家是 winner_correct。

三轮 ToT 上，这两刀各接在哪一行？

原因. Game of 24 和 GSM8K 没有跑 ESC、SR、DPTS。Thought 在 kernel 之前就写好。Pruning 是 queue_fanout_prefills：没留下的 residual 不 queue。Decoding 是之后的 generate_nodes，只发给已经 live 的孩子，步长 32；最后一轮改在 spine 上生成答案。这批 ForkServe+ 是 prefill pruning，192 条里 48 条进 kernel。

调用	这一轮的 thought
第 0 轮 tot_turn_thoughts	Game24：凑成 24 的一对、拆 8×3 或 6×4 、试 $(a \times b) - (c - d)$ 、最大的数当乘数，每条以 Equation：结尾。GSM8K：列方程、倒推、按顺序命名中间量、先列表再运算。
第 1 轮 同上，接在赢家 spine	Game24：用剩下两张牌、把乘积倒过来、加括号、换乘数。GSM8K：检查上一步、换恒等式、从已知重推、丢掉死路。
第 2 轮 不跑 step = 32	Game24：写出用全四张牌的等式。GSM8K：把剩余代数收成最后一行的数字。然后 generate_many 只在已提交的 spine 上出答案。

为什么 prefill 和 decode 不能共用一条 threshold?

原因. 两条分数不在同一个缝里，代码里是一个元组

$(a, b) = (\text{prefill_threshold}, \text{decode_threshold}) = (0.15, 0.45)$ 。Residual 上 loop 是 0.02、illegal 是 0.22，所以 a 只丢 loop。生成前缀上，仍是 illegal 的文本是 0.16，写出 ##### 或 \boxed 是 0.92，所以要 b 才续。只用 b 做 prefill，会把 0.22 的 illegal 挡在 kernel 前；把 a 用到前缀上，0.16 仍然过线，probe 会续满预算。

	loop	illegal	活的文本
prefill 分, residual	0.02	0.22	1.00
decode 分, 生成前缀	0.04	0.16	0.92

MATH-500, $k=16$, $D=1024$		any	时间
full fan-out		20/32	104.1 s
只用 $b=0.45$, 低分不 prefill		13/32	52.1 s
$(a, b) = (0.15, 0.45)$: 只丢 loop, 前缀 $\geq b$ 才续		20/32	72.2 s

决定谁开始, b 决定 should_grow. 分数: PrefillScorer.score_text、DecodeScorer.score_prefix。表: Qwen3-8B, $n=32$, $D=1024$ 。只用 b 是 k16/prefill-all-dead。 (0.15, 0.45) 是 k16/grow@128: probed=128, extended=128。Loop 也 probe 时是 256 次里续写 128 次, 20/32, 74.4 s。

为什么不能把低分全部挡在 prefill 前?

原因. Loop 可以在 kernel 前丢。Illegal text 要先 decode t_0 再打分，否则对的分支也被丢掉。每一行是一对参数：剪什么，以及短 decode 停在哪。设定是 Qwen3-8B，MATH-500 前 32 道 level ≥ 4 ， $D=1024$ ， $\tau=0.45$ ，一半竞赛策略、一半 loop / illegal。

设定	剪	短 decode	$k=4$	$k=8$	$k=16$
full fan-out	不剪	跑满 1024	12/32, 25.3 s	15/32, 52.5 s	20/32, 104.1 s
一刀切 $s<\tau$	低分全丢	留下的跑满	6/32, 19.1 s	11/32, 25.6 s	13/32, 52.1 s
只丢 loop	只丢 loop	其余跑满	11/32, 23.0 s	17/32, 41.5 s	16/32, 71.4 s
只短 decode	不剪	loop 停在 128	10/32, 23.1 s	15/32, 42.3 s	17/32, 74.0 s
probe $t_0=64$	只丢 loop	其余先 64	8/32, 24.0 s	16/32, 42.0 s	18/32, 72.5 s
probe $t_0=128$	只丢 loop	其余先 128	9/32, 24.7 s	13/32, 43.3 s	20/32, 72.2 s
loop 也 probe	不在 kernel 前丢	全部先 128	10/32, 24.9 s	15/32, 43.3 s	20/32, 74.4 s

logs/prune_ablation_scale 与 prune_ablation_grow, 2026-10-01。 $k=16$ 、 $t_0=128$ 的 decode token 从 524288 降到 393216。 温度 0, prefix cache 开, tp=1, 四张 A100-40GB。 格子是 any 和时间。

P/D 分离之后，pruning 少搬了多少？

原因. 精度守得住的切法是 $\tau_{\text{pre}}=0.15$ ：只丢 loop。 $k=16$ 上 any 仍是 16/16，GPU prefill 5008→3848。同一条规则用到 P/D：四条里丢掉 loop，留下的三条仍各带整份 trunk，少搬 $10 \times (256+32)=2880$ ，connector 从 11520 到 8640。

	APC	P/D	P/D, 只丢 loop
Prefill tokens W	11520	11520	8640
Connector tokens X	0	11520	8640
传输时间	0	23.0 ms	17.3 ms
Fan-out	139.0 ms	162.1 ms	121.8 ms

精度：prune_ablation_thresh, Qwen3-8B,

$k=16$, admit 12/16, any 16/16。下表是控制面代价模型，不是 GPU wall clock。10 sessions, $k=4$, $L=256$, $\ell=32$ 。8640=10×3×288。Prefill 12 μs /token, 传输 2 μs /token, abort 0.8 ms。DistServe 的完整引用在背景页。

三轮 ToT 的 expand 为什么比 APC 快?

原因. 三轮合计 192 条 residual, ForkServe+ 只 prefill 48 条。Connector 只搬赢家的 miss。

Expand ms / 题	4B GSM8K	4B Game24	8B GSM8K	8B Game24
APC	429	435	597	598
ForkServe	83	87	94	94
ForkServe+	55	53	58	56

logs/tot_mt, 2026-09-29,
 $n=16, k=4, tp=2, fanout_ms/16$ 。ForkServe+ / APC 是 7.9–10.6 $\times$ 。

我们的方法把端到端效率提高了多少？

原因. 三轮 ToT 上 ForkServe 只做 alias，端到端几乎不降；ForkServe+ 只 prefill 48/192。MATH 与 GSM8K 的对照是 full fan-out 和 ESC base，本方法是 probe 或 τ_{pre} 。

三轮 ToT: $n=16$, $k=4$, $tp=2$, 温度 0。前两轮步长 32，第三轮预算 D 。每题时间是 16 题合计除以 16。decode tok 是 16 题

应用	模型	D	APC	FS	FS+	降幅	做对	decode tok	peak
合计，peak 是 spine。	Game of 24	Qwen3-4B	256	1.61 s	1.64 s	1.13 s 30%	6→8→9/16	3275/3210/ 1928	265→216
	Game of 24	Qwen3-8B	256	2.15 s	2.15 s	1.52 s 29%	9→9→10/16	2943/2853/ 1750	265→216
	GSM8K	Qwen3-4B	512	1.66 s	1.76 s	1.63 s 2%	15→15→15/16	3421/3570/3334	318→275
	GSM8K	Qwen3-8B	512	2.48 s	2.56 s	1.91 s 23%	16→15→15/16	3594/3652/ 2485	318→275

Qwen3-8B, 温度 0, prefix cache 开, $tp=1$, 四张 A100-40GB。MATH-500 是前 32 道 $level \geq 4$, $D=1024$, $\tau=0.45$, 一半竞赛策略、一半 loop / illegal。GSM8K 是 $n=16$, $k=16$, $D=512$, hopeless, ESC 步长 512。

应用	本方法	对照	时间	降幅	any	赢家	prefill	decode
MATH-500	$k=16$, 只丢 loop, probe $t_0=128$	104.1 s	72.2 s	31%	20→20/32	3→3	12320→28736	524288→ 393216
MATH-500	$k=8$, 只丢 loop, probe $t_0=64$	52.5 s	42.0 s	20%	15→16/32	4→3	5592→ 4824	262144→ 196608
GSM8K	ESC, $\tau_{pre}=0.45$, admit 16→8	17.05 s	10.92 s	36%	16→16/16	11→10	5008→ 2768	131072→ 65536
GSM8K	ESC, $\tau_{pre}=0.15$, admit 16→12	17.05 s	14.13 s	17%	16→16/16	11→11	5008→ 3848	131072→ 98304

上表: logs/tot_mt, 2026-09-29。ForkServe+ 的 prefill 是 48/192, pruned_branches=144。下表 MATH:

prune_ablation_scale 的 full 对 prune_ablation_grow 的 grow@ t_0 。 $k=16$ 的 probe 多算了前缀, prefill 从 12320 升到 28736, 时间降在 decode。GSM8K: prune_ablation_thresh。

Game of 24: 为什么端到端能降约 30%?

原因. 这是 Yao 等人 ToT 的原任务。每轮先 propose 一条等式（凑 8×3 、 6×4 ，或把剩下两张牌收成 1），再估价，只提交一条。另外三条 thought 不 prefill。三轮 192 条 residual 里 ForkServe+ 只 prefill 48 条。

	APC	ForkServe+	prefill	做对
Qwen3-4B	1.61 s	1.13 s	48/192	6/16 $\rightarrow$ 9/16
Qwen3-8B	2.15 s	1.52 s	48/192	9/16 $\rightarrow$ 10/16

同一批 tot_mt。4B 降 30%，8B 降 29%。

pruned_branches=144, prefilled_branches=48。

GSM8K: 8B 的三轮策略树为什么降 23%?

原因. 三轮分别是起手策略、沿赢家续写、收成 ##### 数字。起手包括列方程、倒推、先估算再精算。重复 loop 在 prefill 前丢掉, 192 条 residual 里只 prefill 48 条, 端到端从 2.48s 降到 1.91s。

Qwen3-8B, GSM8K, $n=16$	APC	ForkServe	ForkServe+
每题端到端	2.48 s	2.56 s	1.91 s
Expand / 题	597 ms	94 ms	58 ms
做对	16/16	15/16	15/16

logs/tot_mt/qwen3_8b。降幅 23%。

ForkServe+ 的 transfer_tokens 是 1402, prefill 仍是 48/192。

只 admit best case 为什么伤精度?

原因. 在 batch size = 8, app 不按分数留一半, 把对的分支也丢掉了。

$k=16, \tau=0.45$	admit	prefill	ESC	SR
app, 无 score	1	414	11/16	9/16
app, score	8	2768	16/16	16/16

logs/prune_ablation_wide/all.json。GSM8K, Qwen3-8B,

$n=16$ 。